

FREE GUIDE

FOUNDER'S LINUX STARTER PACK

LINUX FOR NON- TECHNICAL FOUNDERS

From zero terminal knowledge to running a real server with AI.

What is Linux

Your First Server

Core Commands

Deploy an App

AI Management

Security Basics

terminal - bosh

\$ ssh root@your-server-ip

Welcome to Ubuntu 22.04 LTS

\$ sudo apt update && sudo apt upgrade -y

Your server. Your rules.

\$ _

BEFORE YOU START

Why Founders Need to Understand Linux

Every product you build eventually runs on a server. And almost every serious server in the world runs Linux. Not Windows. Not macOS. Linux.

Most non-technical founders never touch it because managed platforms like Vercel, Railway, and Supabase abstract it away. That abstraction is useful - until it is not. Until you need more control, lower costs, or the ability to understand what is actually happening underneath your product.

This guide is for founders and vibecoders who are ready to stop treating the server as a black box. You do not need years of systems engineering experience. You need a clear mental model, a few core commands, and the ability to use AI as your terminal co-pilot.

By the end of this guide you will have set up a real Linux server, deployed something on it, and know how to use Claude or ChatGPT to manage it going forward without memorising every command.

WHAT THIS GUIDE COVERS

- Section 01 - What Linux Actually Is
- Section 02 - Setting Up Your First Server
- Section 03 - Core Commands Every Founder Needs
- Section 04 - Deploying a Real App on Your Server
- Section 05 - Using AI to Manage Your Server
- Section 06 - Security Basics Before You Go Live

"Technical autonomy is not about knowing everything. It is about knowing enough to direct the machine - and knowing how to ask for help when you do not."

01

WHAT LINUX ACTUALLY IS

A mental model, not a memorisation exercise.

The Simple Version

Linux is an operating system - the same way Windows and macOS are operating systems. It manages your hardware, runs your programs, and handles everything happening on a computer. The difference is that Linux was built to run servers, not desktops. It is lean, fast, stable, and free. That is why 96% of the world's servers run it.

When you deploy to Vercel, Railway, or any cloud provider, your code is running on a Linux server somewhere. You just never had to look at it. This guide changes that.

THE THREE THINGS YOU NEED TO UNDERSTAND

The Kernel

The core of Linux. It sits between your software and your hardware and manages everything - memory, processes, storage, network. You never interact with it directly, but everything you do goes through it.

The Terminal

Your interface to Linux. Instead of clicking icons, you type commands. It feels intimidating at first. It becomes the fastest way to work once you know 15-20 commands. This guide gives you the ones that matter.

Distributions

Linux comes in different flavours called distributions (distros). Ubuntu is the most beginner-friendly and the most widely supported. This guide uses Ubuntu 22.04 LTS. LTS means Long Term Support - it will receive security updates for 5 years. Use it.

WHY NOT JUST STAY ON VERCEL OR RAILWAY?

Managed platforms are excellent for getting started and for many production workloads. But there are situations where owning your server makes more sense:

MANAGED PLATFORM	YOUR OWN LINUX SERVER
Simple deployment, no config	Full control over every config
Pricing scales up fast	\$5-10/month regardless of traffic

Limited environment access	Install anything, run anything
Vendor lock-in risk	Portable - move providers anytime
Great for early MVPs	Better for scale and cost control

TIP

You do not have to choose one or the other permanently. Many founders use managed platforms for their main product and a VPS for internal tools, background workers, or experiments. Understanding Linux gives you the option - not the obligation.

02

SETTING UP YOUR FIRST SERVER

From zero to a live server in under 20 minutes.

STEP 1 - CHOOSE A VPS PROVIDER

A VPS (Virtual Private Server) is a server you rent by the month. You get root access to a Linux machine hosted in a data center. These are the most beginner-friendly providers:

DigitalOcean	droplets.digitalocean.com	Cleanest UI for beginners. \$6/month for a basic Droplet. Excellent documentation.
Hetzner	hetzner.com/cloud	Best value for money. €4/month for equivalent specs. Popular in Europe.
Vultr	vultr.com	Good global coverage. \$6/month. Simple interface.
Linode (Akamai)	linode.com	Reliable, well-documented. \$5/month starting.

TIP

When creating your server, always choose Ubuntu 22.04 LTS as the operating system. Choose the cheapest plan to start - 1 CPU, 1GB RAM is enough to run most small apps. You can upgrade later.

STEP 2 - CONNECT TO YOUR SERVER WITH SSH

SSH (Secure Shell) is how you connect to your server from your laptop. Think of it as a secure remote control. When you create your VPS, your provider gives you an IP address - a series of numbers like 143.198.123.45. That is your server's address.



bash

Open your terminal (Mac/Linux) or PowerShell (Windows)

\$ ssh root@YOUR-SERVER-IP

Example:

\$ ssh root@143.198.123.45

First time connecting - type yes when asked

Are you sure you want to continue connecting? yes

You are now inside your server

```
root@ubuntu-server:~$ _
```

TIP

If your provider offers SSH key authentication during setup, use it. It is more secure than a password. Most providers show you how to add your SSH key during the server creation process.

STEP 3 - FIRST THINGS TO DO ON A NEW SERVER

Run these commands every time you set up a new server. In order.

```
● ● ● bash

# Update the package list and upgrade installed packages
$ sudo apt update && sudo apt upgrade -y

# Create a new user (replace "yourname" with your actual name)
$ adduser yourname

# Give the new user admin privileges
$ usermod -aG sudo yourname

# Switch to your new user
$ su - yourname

# Verify sudo works
$ sudo whoami

root
```

IMPORTANT

Never work as root for day-to-day tasks. Root has unlimited power - one wrong command can delete your entire server. Create a user account and use that instead. Use sudo only when a command specifically requires it.

03

CORE COMMANDS EVERY FOUNDER NEEDS

You do not need to memorise 200 commands. You need these 20.

NAVIGATION - MOVING AROUND

<code>pwd</code>	Print working directory - shows where you are right now
<code>ls</code>	List files and folders in the current directory
<code>ls -la</code>	List all files including hidden ones, with details
<code>cd folder-name</code>	Change into a directory
<code>cd ..</code>	Go up one directory level
<code>cd ~</code>	Go to your home directory from anywhere

FILES AND FOLDERS

<code>mkdir folder-name</code>	Create a new folder
<code>touch filename.txt</code>	Create a new empty file
<code>cp file.txt copy.txt</code>	Copy a file
<code>mv file.txt /new/path/</code>	Move or rename a file
<code>rm filename.txt</code>	Delete a file permanently
<code>rm -rf folder-name</code>	Delete a folder and everything inside it - use carefully
<code>cat filename.txt</code>	Print file contents to the terminal
<code>nano filename.txt</code>	Open a file in the nano text editor

SYSTEM AND PROCESSES

<code>top</code>	Live view of CPU and memory usage - press Q to quit
<code>htop</code>	Better version of top - install with: <code>sudo apt install htop</code>

<code>ps aux</code>	List all running processes
<code>kill PID</code>	Stop a process by its ID number
<code>df -h</code>	Show disk space usage in human-readable format
<code>free -h</code>	Show memory (RAM) usage

NETWORK AND PERMISSIONS

<code>curl https://example.com</code>	Make an HTTP request from the terminal
<code>wget https://file-url.com/file</code>	Download a file from the internet
<code>chmod 755 filename</code>	Change file permissions
<code>chown user:group filename</code>	Change file ownership
<code>ufw status</code>	Check firewall status
<code>sudo ufw allow 80</code>	Allow traffic on port 80 (HTTP)

TIP

If you forget what a command does, type: `man command-name` (e.g. `man ls`). This opens the manual page. Press `Q` to exit. Or just ask your AI - "what does the `rm -rf` command do in Linux?"

04

DEPLOYING A REAL APP ON YOUR SERVER

A Node.js app from your laptop to a live URL - step by step.

This section walks through deploying a Node.js application. The same principles apply to Python (Flask/FastAPI), PHP, or any other language. We will use Nginx as a reverse proxy - it sits in front of your app and handles incoming web traffic - and PM2 to keep your app running even after you log out.

INSTALL NODE.JS AND NPM

```
● ● ● bash

$ curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
$ sudo apt install -y nodejs
$ node --version
v20.x.x
$ npm --version
10.x.x
```

INSTALL NGINX (REVERSE PROXY)

```
● ● ● bash

$ sudo apt install nginx -y
$ sudo systemctl start nginx
$ sudo systemctl enable nginx
# Visit your server IP in a browser - you should see the Nginx welcome page
```

UPLOAD YOUR APP TO THE SERVER

```
● ● ● bash

# Option A - clone from GitHub (recommended)
$ git clone https://github.com/yourusername/your-app.git
$ cd your-app
$ npm install

# Option B - copy files from your laptop using SCP
$ scp -r ./your-app yourname@YOUR-SERVER-IP:/home/yourname/
```

INSTALL PM2 AND START YOUR APP

```
● ● ● bash

$ sudo npm install -g pm2

$ pm2 start app.js --name "my-app"

# Replace app.js with your entry file

$ pm2 status

# You should see your app listed as online

$ pm2 startup

# Run the command it gives you - this makes your app restart on reboot

$ pm2 save
```

CONFIGURE NGINX TO POINT TO YOUR APP

```
● ● ● bash

$ sudo nano /etc/nginx/sites-available/my-app

# Paste this configuration (replace YOUR-SERVER-IP and port):

server {

listen 80;

server_name YOUR-SERVER-IP;

location / {

proxy_pass http://localhost:3000;

proxy_http_version 1.1;

proxy_set_header Upgrade $http_upgrade;

proxy_set_header Connection "upgrade";

proxy_set_header Host $host;

}

}

# Save the file (Ctrl+X, then Y, then Enter)

$ sudo ln -s /etc/nginx/sites-available/my-app /etc/nginx/sites-enabled/

$ sudo nginx -t

$ sudo systemctl reload nginx

# Visit your server IP - your app should now be live
```

TIP

To add a real domain name instead of an IP address, point your domain DNS A record to your server IP, then update the `server_name` field in your Nginx config from `YOUR-SERVER-IP` to `your-domain.com`. Then add SSL with: `sudo apt install certbot python3-certbot-nginx && sudo certbot --nginx -d your-domain.com`

05

USING AI TO MANAGE YOUR SERVER

You do not need to memorise everything. You need to know what to ask.

The single biggest shift in server management for non-technical founders is this: you do not need to know the exact command. You need to know what you want to happen, and you need to give the AI enough context to help you do it safely.

The mental model is the same as managing AI in your vibe-coding builds - you are the senior, the AI is your junior. Give it context. Tell it your OS, your setup, your goal. Review what it gives you before running it on a live server.

THE CONTEXT BLOCK FOR SERVER MANAGEMENT

Start every AI server management session with this block. Fill it in for your setup.

I am managing a Linux server with the following setup: OS: Ubuntu 22.04 LTS Provider: [your VPS provider] Server IP: [your IP] Running: [e.g. Node.js app on port 3000, managed by PM2] Web server: Nginx as reverse proxy User: [your username] with sudo access I am not a systems engineer. Please explain what each command does before I run it, and flag any commands that could cause problems if run incorrectly.

AI PROMPT LIBRARY FOR SERVER MANAGEMENT

SITUATION

My app crashed and I need to see what went wrong.

My Node.js app managed by PM2 has stopped. How do I check the logs to see what caused it to crash? My setup: [paste context block]

SITUATION

I need to check how much disk space is left.

Show me how to check disk space usage on my Ubuntu 22.04 server and identify which directories are using the most space.

SITUATION

My server is running slow - I need to diagnose it.

My server feels slow. Walk me through diagnosing the issue - I want to check CPU, memory, and which processes are consuming the most resources. My setup: [paste context block]

SITUATION

I need to set up automatic backups.

I want to automatically back up my /home/yourname/app directory every night at 2am and store the last 7 backups. Show me how to do this on Ubuntu 22.04 using cron.

SITUATION

I got an error I do not understand.

I ran this command on my Ubuntu server and got this error: [paste the exact error message]. My setup is: [paste context block]. What does this mean and how do I fix it safely?

SITUATION

I want to update my app on the server.

I have pushed new code to GitHub and want to pull it onto my server and restart the app with zero downtime. My setup: Node.js app managed by PM2, code in /home/yourname/app. Walk me through it step by step.

IMPORTANT

Always test AI-suggested commands on a staging server or a fresh VPS before running them on a production server. If you only have one server, ask the AI: "Is this command reversible? What is the worst case if something goes wrong?"

06

SECURITY BASICS BEFORE YOU GO LIVE

A server without these is not a server - it is an open door.

Security does not require an advanced degree. The basics cover 90% of common attacks on small servers. Do all of these before you point a domain at your server or tell anyone the IP address.

DISABLE ROOT LOGIN OVER SSH

This prevents automated bots from brute-forcing your root password. They try root because it exists on every server. Remove it as an SSH option.

● ● ● bash

Edit the SSH configuration file

```
$ sudo nano /etc/ssh/sshd_config
```

Find this line and change yes to no:

```
PermitRootLogin no
```

Save and restart SSH

```
$ sudo systemctl restart sshd
```

Now only your named user account can log in

SET UP THE UFW FIREWALL

UFW blocks all incoming traffic except what you explicitly allow. SSH, HTTP (80), and HTTPS (443) are the minimum. Everything else stays closed.

● ● ● bash

Install and configure UFW (Uncomplicated Firewall)

```
$ sudo apt install ufw -y
```

Allow SSH - do this FIRST or you will lock yourself out

```
$ sudo ufw allow OpenSSH
```

Allow web traffic

```
$ sudo ufw allow 80
```

```
$ sudo ufw allow 443
```

Enable the firewall

```
$ sudo ufw enable
$ sudo ufw status
# Should show Active and your allowed ports
```

KEEP YOUR SERVER UPDATED

Most server compromises exploit known vulnerabilities in outdated packages. Regular updates close those gaps before attackers can use them.

```
● ● ● bash
# Run this regularly - monthly at minimum
$ sudo apt update && sudo apt upgrade -y
# Set up automatic security updates
$ sudo apt install unattended-upgrades -y
$ sudo dpkg-reconfigure --priority=low unattended-upgrades
# Choose Yes when prompted
```

USE SSH KEYS, NOT PASSWORDS

Passwords can be guessed. SSH keys cannot be brute-forced in any practical timeframe. If you only do one security step, make it this one.

```
● ● ● bash
# On your LOCAL machine (not the server), generate a key pair:
$ ssh-keygen -t ed25519 -C "your-email@example.com"
# Copy your public key to the server:
$ ssh-copy-id yourname@YOUR-SERVER-IP
# Then disable password authentication on the server:
$ sudo nano /etc/ssh/sshd_config
PasswordAuthentication no
$ sudo systemctl restart sshd
```

SECURITY CHECKLIST - BEFORE YOU GO LIVE

- Root SSH login is disabled
- A named user account with sudo access is created
- SSH key authentication is set up

- Password authentication over SSH is disabled
- UFW firewall is active and only allows ports 22, 80, 443
- Server packages are up to date
- Automatic security updates are enabled
- App is not running as root
- Environment variables and secrets are not hardcoded in files
- SSL certificate is installed if using a domain

FINALLY

You Now Have a Server. Keep Building.

Technical autonomy is not about knowing everything. It is about knowing enough to direct the machine - and knowing how to ask the right questions when you do not.

You have just gone from zero to a live, secured Linux server with a real app running on it. That puts you ahead of most non-technical founders who are still entirely dependent on managed platforms for everything.

From here, the best way to get better is to use it. Break something. Fix it. Ask the AI what happened. Break it again. That cycle - more than any guide - is what builds real confidence with a server.

This guide is part of The Weekly Architecture Audit - a newsletter where I break down the technical and strategic decisions behind building real products. If this was useful, subscribe for more.

Thosyn Pax

Product Architect | The Product Lab

Enjoyed this guide?

The VibeCoder's Playbook goes deeper on building full products with AI - 8 phases, 6 templates, field-tested. Available on Gumroad and Selar.

PODCAST The Product Lab
Conversations

NEWSLETTER The Weekly
Architecture Audit

YOUTUBE Product Lab by
Thosyn Pax

WEBSITE thosynpax.com/lab